Objective 2 - Uttarakhand

Simulation, algorithms and the reasons behind them

PCM capsule storage in a 50 L solar water tank, four climate regimes (K = 4). Version of 9 November 2026.

> **In one paragraph.** For each of Uttarakhand's four climate regimes from Objective 1, we searched for the capsule size, amount of PCM, pump flow and arrangement that give a 50 L solar tank the most extra hot water at 50 C between 18:00 and 09:00, compared with the same tank holding water only. A physics simulator evaluates every design over full years of real weather; a sequence of design-of-experiments, surrogate and multi-objective optimisation steps decides which designs to simulate; a pre-declared rule picks the final design on separate years; and a one-time test on two untouched years (2024-2025) confirms it. PCM helps in regimes 0-2 (56-76 kWh/yr more on the test years); in regime 3 (cold, high altitude) it does not, and the recommendation there is a water-only tank.

1. The problem being solved 3
   1.1 What is fixed and what is designed 3
   1.2 How the ten weather years are used 3
2. What is included in the simulation 3
   2.1 The parts of the model 3
   2.2 What the simulator reports (the metrics) 4
   2.3 Checks that the simulator is right 4
   2.4 What is deliberately not modelled 4
3. The algorithms, in the order they run 5
4. Each algorithm: why this one and not another 6
   4.1 Grey-box simulator (instead of CFD or a commercial tool) 6
   4.2 Enthalpy method for the PCM 6
   4.3 Multi-node (stratified) tank, 10 layers 6
   4.4 Linear-implicit (backward Euler) time stepping at 150 s 7
   4.5 Paired baseline and monotone (PCHIP) interpolation 7
   4.6 Explore / select / test split of the weather years 7
   4.7 Taguchi L9 orthogonal array with ANOVA (screening) 8
   4.8 Scrambled Sobol sequence for the initial DOE 8
   4.9 Surrogate models: XGBoost, quadratic RSM and an NN ensemble 9
   4.10 Repeatability band (WP6) instead of p-values 9
   4.11 NSGA-II, active learning and the box-vertex step 10
   4.12 Dense grid with hypervolume and epsilon indicators (checking the optimiser) 10
   4.13 Selection rule (S0 - S1 - S2, then lowest PCM mass) 11
   4.14 One-shot held-out test, ablations and controls 11
   4.15 CFD check: OpenFOAM, enthalpy-porosity, buoyantPimpleFoam 12
5. Running Objective 2 faster: parallel tasks and CPU use 13
   5.1 The machine and where the time goes (measured) 13
   5.2 Ways to make it faster 13
   5.3 What does not help, and why 14
   5.4 How method 2 would be written (proposed code, not yet in the repository) 14
   5.5 What can run in parallel 14
   5.6 Recommended: the pipeline's own parallel runner 15
   5.7 The same steps by hand in PowerShell (12 pairs) 15

5.8 OpenFOAM cases in parallel (safe now) 16
6. The final output of Objective 2 16
7. Summary table 17

1. The problem being solved

Objective 1 grouped Uttarakhand's weather points into **4 climate regimes** and, for each regime, ranked the PCMs and gave a **top-3 shortlist**. Objective 2 takes these as fixed inputs and designs the storage unit.

Regime	Medoid point	Points	Mains water	Objective 1 top-3 PCMs
0 (warm)	UKP_0014	10	21.8 C	PureTemp 58, n-Octacosane (C28), PlusICE A58
1	UKP_0029	23	18.6 C	PureTemp 58, n-Octacosane (C28), PlusICE A58
2	UKP_0025	9	17.0 C	PureTemp 58, savE OM55, n-Hexacosane (C26)
3 (cold, high altitude)	UKP_0041	3	7.4 C	PureTemp 53, n-Hexacosane (C26), Myristic acid (C14)

4 regimes x 3 PCMs = **12 (regime, PCM) pairs**. Every pair is optimised on its own.

1.1 What is fixed and what is designed

Fixed (from configs/system_config_shared.yaml)	Designed (configs/design_space.yaml)
50 L vertical tank, height/diameter 2, 5 cm insulation, U = 0.8 W/m2K	Capsule inner radius: 10-40 mm
1.5 m2 flat-plate collector, F_R(tau alpha) = 0.75, F_R U_L = 4.5 W/m2K	PCM volume fraction of the tank: 2-30 % (sets the number of capsules)
Aluminium capsule shell 0.8 mm, k = 205 W/mK	Pump flow rate: 0.010-0.050 kg/s
Delivery target 50 C; limits: water 75 C, PCM 65 C	Arrangement: single-layer, staggered or radial
Hot-water demand 300 L/day, Uttarakhand profile, local time, peak 07:30	PCM: one of the regime's three Objective 1 PCMs

1.2 How the ten weather years are used

Role	Years	Used for
Explore	2016-2020	Screening, DOE, surrogates, optimisation, dense grid, ablations
Select	2021-2023	Choosing the final design per regime
Test (held-out)	2024-2025	One single evaluation of the chosen designs, then locked

Hourly NASA POWER data for each regime's medoid point; all 40 point-years passed the acceptance rule (at least 98 % valid hours), 29 February is dropped so every year has 8,760 hours.

2. What is included in the simulation

The simulator is a **grey-box model**: each part is a physics equation with a small number of physical parameters, not a fitted black box. It runs one full year in about 1 second, which is what makes searching thousands of designs possible. Entry point: `src/simulation/run_case.py: run_case(...)`; `pcm_name=None` gives the water-only tank with the same design.

2.1 The parts of the model

Part	Model used	File
Solar collector	Hottel-Whillier-Bliss: Q = A [F_R(tau alpha) I - F_R U_L (T_in - T_amb)]. Pump off below 10 W/m2, so no heat loss at night	collector_model.py
Tank	10 stacked water layers (stratified). Draw takes hot water from the top and mains water enters at the bottom (plug flow, implicit upwind). Collector water returns to the layer of equal temperature. Conduction between layers, heat loss through the wall, and mixing of any inverted layers (energy-conserving)	tank_model_multinode.py
PCM capsules	Enthalpy method: h(T) piecewise (solid, melting band Tm +/- 1 K, liquid), melt fraction f = clip((h - h_s)/L, 0, 1). The aluminium wall's heat capacity is carried in the PCM node	capsule_enthalpy.py

Part	Model used	File
PCM mass	From the INNER sphere (D - 2 t_wall); the whole capsule displaces water	capsule_mass.py
Water to PCM heat flow	Three resistances in series: 1/UA = 1/(h_w A) + R_wall + R_pcm. h_w from the Wakao-Kaguei packed-bed correlation Nu = 2 + 1.1 Re^0.6 Pr^1/3; R_pcm = r/(5 k_eff), with k_eff = 2 k once the PCM is half melted (natural convection in the melt)	heat_transfer.py
Hot-water demand	Hourly profile of 300 L/day, spread evenly inside each hour, in local time (IST)	demand_profile.py
Mains water	One temperature per regime from Objective 1 (21.8 / 18.6 / 17.0 / 7.4 C)	configs/states/uttarakhand.yaml
Safety shield	Charging is bypassed when the top layer reaches 72 C or the PCM reaches 62 C (3 C below the 75 / 65 C limits)	tank_model_multinode.py
Pump and pressure drop	Ergun equation; reported but flagged as not meaningful	hydraulic_model.py
Time stepping	150 s step, linear-implicit (backward Euler) water layers, PCM updated with the new water temperature; 6 sub-steps inside the melting band	tank_model*.py
Energy check	E_collector + E_initial = E_load + E_loss + E_final + residual; residual must be below 0.1 %	energy_balance.py

2.2 What the simulator reports (the metrics)

Metric	Meaning
E_del (column names end in 45, value computed at 50 C)	Energy delivered in the draws up to the 50 C target, above the mains temperature
g_window (main objective)	E_del from 18:00 to 09:00 of the PCM tank minus that of the water-only tank at the same flow and year (kWh/yr)
g_annual	The same over the whole year
d_overheat	Shield hours of the PCM tank minus those of the water-only tank (lower is better)
tau_serve	Hours from 18:00 with no sun until the outlet first drops below 50 C
t_pcm_max, t_water_max	Peak temperatures; a design is feasible only if the PCM stays at or below 65 C and the water at or below 75 C in every year

2.3 Checks that the simulator is right

- **Time step:** 150 s against 75 s and 37.5 s changes results by at most 0.029 %; energy balance residual at most 0.076 % (`results/g2_timestep_convergence_150_75_37p5.csv`).
- **Phase-4 verification gates 1-5:** energy conservation, zero-sun and zero-demand behaviour, physical limits, solar fraction against published values (passes with a caveat: below the published band, reported and not tuned), PCM behaviour (`results/phase4_verification_o1_pcms.txt`).
- **Number of tank layers:** against the project's OpenFOAM draw case, 10 layers give 22.5 L at or above the target where the CFD gives 27 L and one fully mixed layer gives 0.
- **CFD (OpenFOAM) capsule check:** the first Uttarakhand case (regime 0, PureTemp 58) melts to 90 % in 71 min in CFD; the simulator predicts 145 min, so the capsule model is conservative (it charges more slowly than the CFD).

2.4 What is deliberately not modelled

Supercooling, PCM volume change on melting, a backup heater, pump energy, a flow-dependent collector F_R and a detailed melt front inside each capsule. These are listed in `docs/05_LIMITATIONS_AND_OPEN_ITEMS.md`; none of them can be added without data that the project does not have.

3. The algorithms, in the order they run

Each step has its own script in `scripts/`; `pipeline.py` runs them in order. The surrogate models only suggest which designs to try: **every number reported is a real simulator run**.

Step	Algorithm	Purpose	Size for Uttarakhand
G7	One-at-a-time sensitivity	First look at which variable matters	4 regimes, explore years 2018-2019
S1	Taguchi L9 orthogonal array + ANOVA	Screen the 4 variables	9 designs x 5 years x 12 pairs
S2	Scrambled Sobol sequence (quasi-Monte Carlo DOE)	Spread designs evenly over the box	128 per pair, 1,536 total (1,452 simulated, 84 not buildable)
S3	Surrogates: XGBoost, quadratic RSM, NN ensemble; chosen by Spearman rank correlation in cross-validation	A fast stand-in for the simulator inside the optimiser	XGBoost won 11 pairs, RSM 1
WP6	Repeatability band (combined uncertainty)	How big a gain must be to count as real	About 22 / 19 / 20 / 6 kWh for regimes 0-3
S4	NSGA-II multi-objective genetic algorithm	Find the best trade-offs (Pareto front)	Population 100, 100 generations, 3 seeds
S5	Active learning by committee disagreement, plus the box-vertex step	Correct the surrogate where it is unsure	5 rounds, 24 corners, 3 more rounds; 193 min for 12 pairs
S6	Dense regular grid + hypervolume and epsilon indicators	Check NSGA-II did not miss a better region	720 designs per regime (rank-1 PCM)
S7	Band-based lexicographic selection rule	Pick one design per regime on the select years	4 regimes
S8	One-shot held-out test with a lock file	Honest final check on unseen years	2024-2025, run once
G11+	Ablations, negative and positive controls, diagnostics	Show the gain comes from the PCM physics	4 regimes
CFD	OpenFOAM enthalpy-porosity, buoyantPimpleFoam	Independent check of the capsule model	10 capsule + 4 animation + 12 tank cases (1 run so far)

The sections below explain each step: what it is, why it was chosen, and which alternatives were considered and why they were not used.

4. Each algorithm: why this one and not another

4.1 Grey-box simulator (instead of CFD or a commercial tool)

What it is and what we did

A physics model built from standard equations (Section 2), written in Python and run for every design and every weather year.

Why we used it

- It is fast: about 1 s per simulated year, so the 1,536 DOE designs, the active learning, the 2,880 dense-grid designs and all checks (tens of thousands of simulated years in total) were possible.
- Every part is a standard, citable equation (Hottel-Whillier-Bliss, Wakao-Kaguei, enthalpy method), so each result can be traced to physics.
- It is the same code as Rajasthan, so the two states are compared by the same rules.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Full CFD (OpenFOAM, Fluent)	One capsule takes minutes to hours and one tank-year would take weeks; impossible for thousands of designs. Used instead as an independent check (Section 4.15).
TRNSYS / EnergyPlus	Commercial or heavy tools; PCM capsule beds need custom component types anyway, and they cannot be called thousands of times from a Python optimiser easily.
Pure machine learning model trained on data	No measured data exist for these tanks; a model must be based on physics to be trusted outside measured conditions.

4.2 Enthalpy method for the PCM

What it is and what we did

Each capsule group stores enthalpy h; temperature and melt fraction follow from h through a piecewise function (solid, a melting band of Tm +/- 1 K, liquid).

Why we used it

- Latent heat is conserved exactly: no energy is lost or created when the PCM crosses its melting point.
- The function can be inverted exactly, so the model never fails to converge at the melting point.
- It needs only the properties Objective 1 provides (Tm, L, cp, k, density).

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Apparent heat capacity method	Puts the latent heat into a peak of cp; with a 150 s step the temperature can jump over the narrow peak and lose latent heat.
Stefan-problem front tracking	Follows the moving melt front exactly, but needs a mesh inside every capsule; far too slow for a yearly simulation of 4-55 capsules.
Enthalpy-porosity (as in CFD)	Needs the flow inside the melt to be solved; that is what the CFD check does, not the yearly model.

4.3 Multi-node (stratified) tank, 10 layers

What it is and what we did

The tank is 10 layers; hot water stays on top, mains water enters at the bottom, collector water returns to the layer of the same temperature.

Why we used it

- Real tanks are stratified: the user draws from the hot top. A single mixed node gave 0 litres at the target in the CFD draw case, 10 layers gave 22.5 L, the CFD 27 L.
- 10 layers is the usual choice in solar-tank models; going to 20 layers changed the useful energy by less than 1 %, and that effect is included in the uncertainty band.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Single mixed node	Kept only as a regression reference; it cannot represent hot water at the top, so it underestimates delivery at 50 C.
20 or 40 layers	Changes results by under 1 % but doubles the run time; the difference is counted in the band instead.
CFD tank	Weeks per year of weather; used only to check the stratification (tank cases in the OpenFOAM folder).

4.4 Linear-implicit (backward Euler) time stepping at 150 s

What it is and what we did

The water layers are solved implicitly in closed form each step; the PCM is then updated with the new water temperature (semi-implicit coupling), with 6 extra sub-steps inside the melting band.

Why we used it

- The water node is the stiff part (large heat-transfer coefficient, small mass); an implicit step stays stable at 150 s.
- No iterations are needed, so a year runs in about 1 s.
- Checked: halving the step twice changes the results by at most 0.029 %. Rajasthan showed that 300 s fails this check, so 150 s is used.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Explicit Euler	Would need a step of a few seconds to stay stable; about 50 times slower.
Adaptive Runge-Kutta (scipy solve_ivp)	Accurate, but the sharp changes at the melting point and at pump on/off make it take tiny steps; slower and harder to verify energy conservation.
Fully implicit with Newton iterations	More accurate in theory, but the measured error at 150 s is already 0.03 %; extra cost for no gain.

4.5 Paired baseline and monotone (PCHIP) interpolation

What it is and what we did

Every PCM design is compared with a water-only tank at the same flow, weather year, demand, shield and tank model. The water-only tank is simulated on a 17-point flow grid and interpolated with PCHIP; the held-out test uses exact runs.

Why we used it

- The question is 'does PCM help?'. Pairing removes everything that is not the PCM: weather, demand and flow affect both tanks the same way.
- The year-to-year variation of the gain is much smaller than the variation of the absolute energy, so smaller real effects can be seen.
- PCHIP keeps the curve monotone with no overshoot between grid points; its error is measured in tests/test_design_space.py.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Compare absolute energy only	A design with a higher flow would look better simply because of the flow, not the PCM.
Single fixed water-only reference	Mixes the effect of flow with the effect of PCM.
Linear or cubic-spline interpolation	Linear is less accurate between points; a cubic spline can overshoot and create false gains.

4.6 Explore / select / test split of the weather years

What it is and what we did

Three separate groups of years: search on 2016-2020, choose on 2021-2023, test once on 2024-2025.

Why we used it

- Searching and choosing on the same years makes the result look better than it is (selection bias). Choosing on separate years and then testing on years never touched gives an honest number.
- The test years are below-average sunshine years in every regime, so the test is a conservative check.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Use all 10 years for everything	No honest check of the final result is left.
Random k-fold over years	Weather years are not interchangeable samples, and the final test must stay unseen; a random split would let the test years influence the choice.
Test on 2011-2015 (as Rajasthan v3)	Rajasthan used those because its 2024-2025 test had already been spent. Uttarakhand had no earlier test, so the recent years were used and no older data were downloaded.

4.7 Taguchi L9 orthogonal array with ANOVA (screening)

What it is and what we did

Nine designs chosen so that every level of each of the 4 variables appears equally often with every level of the others (levels: radius 10/25/40 mm, PCM 2/16/30 %, flow 0.010/0.030/0.050 kg/s, 3 arrangements). Each is simulated for 5 years; ANOVA gives the share of variation due to each variable.

Why we used it

- 9 runs instead of 81 for the full 3^4 factorial, but every variable is still estimated fairly.
- It is the standard method in the PCM-tank papers the project follows (Chen et al. 2025), so the results are comparable.
- Screening only: it shows which variables matter; it does not choose the design.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Full factorial (81 runs)	9 times the cost for a screening step whose only job is a first ranking.
Plackett-Burman	Two-level only; cannot see the curvature that PCM fraction and radius show.
Morris method or Sobol sensitivity indices	Need hundreds of runs per pair for a screening result; the S2 DOE and the ablations already give the detailed picture.

4.8 Scrambled Sobol sequence for the initial DOE

What it is and what we did

128 points (2^7) spread over the 4-variable box with a scrambled Sobol low-discrepancy sequence (seed 20261026). The same points are used for all 12 pairs so the pairs are comparable. Every point is simulated for 5 explore years.

Why we used it

- Sobol points fill the space more evenly than random points, so the surrogates learn the whole box with fewer runs.
- Scrambling keeps the even spread but removes the regular patterns of the plain sequence.
- A power of two (128) keeps the balance property of the Sobol sequence.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Plain random (Monte Carlo) sampling	Leaves gaps and clusters; needs more points for the same coverage.
Latin hypercube sampling	Even in each variable on its own, but not necessarily in combinations; Sobol is at least as good for 4 variables and is deterministic with a seed.
Regular grid	Too many points for the same density (used later only as a check, Section 4.12).

4.9 Surrogate models: XGBoost, quadratic RSM and an NN ensemble

What it is and what we did

Three model families are trained on each pair's DOE: XGBoost (gradient-boosted trees, 300 trees, depth 3), a full quadratic response surface with ridge regularisation (RSM), and an ensemble of 5 small neural networks (32-32). Each is checked with 5-fold cross-validation over designs and leave-one-year-out cross-validation. The family with the best Spearman rank correlation between predicted and simulated rankings is used for that pair (ties within 0.02 broken by lower error). Result: XGBoost in 11 pairs, RSM in r0_p2. The same three families and rule are used in Rajasthan (XGBoost 6 of 9 pairs, RSM 3).

Why we used it

- **The surrogate does not decide anything.** It is only used inside NSGA-II to propose designs; every candidate is then re-simulated, and the final choice and all reported numbers come from the simulator. A different surrogate could change which designs are tried first, not what the results are, and the dense-grid check (Section 4.12) shows the front found is the true front within the band.
- The choice between the three is made by data, not preference. The rule asks which model ranks designs correctly, which is what an optimiser needs.
- XGBoost handles the categorical arrangement variable and the sharp changes caused by the safety shield and by the geometry limits (where a smooth model struggles); RSM wins where the response is smooth (r0_p2); the NN ensemble gives an uncertainty estimate.
- Three different families are also needed for the active-learning step: their disagreement shows where the surrogate is unsure (Section 4.11).
- The methods follow the cited literature (Yan et al. 2025 for XGBoost, Li et al. 2020 for RSM) and keep Uttarakhand identical to Rajasthan.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Gaussian-process regression (Kriging)	The usual engineering choice and a fair alternative. It was not in the project recipe. With one categorical variable and sharp shield/geometry effects, a standard (stationary) Kriging model needs special kernels. It could be added as a fourth family, and the same Spearman rule would decide; the final results would still come from the simulator.
Radial basis functions	Interpolate the data exactly, so they also fit the year-to-year noise; no built-in uncertainty.
Random forest / Extra Trees	Used by the retired legacy chain (Extra Trees). XGBoost is the boosted form of the same tree idea and is usually more accurate on small tabular data sets like this one.
Support vector regression	Needs careful tuning of kernel and margins for each of 12 pairs and 3 targets; no clear advantage at this data size.
Deep neural networks	About 128-200 training points per pair is too few for a large network; the small NN ensemble is the sensible size.
Stacked ensemble (Tamil Nadu)	Combines several models; adds complexity, and because the surrogate does not decide the result, the gain would not change the final designs.

> **Why XGBoost is kept.** It is chosen automatically by the validation rule, it never produces a reported number, and keeping the same recipe as Rajasthan keeps the two states comparable. Changing it now would also mean re-running the optimisation and selection after the one-time held-out test has already been used, which would break the honesty of that test.

4.10 Repeatability band (WP6) instead of p-values

What it is and what we did

Before a gain is called real it must be larger than a band that combines four measured sources of uncertainty: band = sqrt((year sd / sqrt 5)^2 + (time-step effect)^2 + (10 vs 20 layers)^2 + (PCM property effect, latent heat +/-10 %, conductivity +/-20 %)^2). Measured on 4 designs per regime: about 22 / 19 / 20 / 6 kWh/yr for regimes 0-3.

Why we used it

- It includes the model's own uncertainty, not only the weather; a p-value would ignore the time step, the number of layers and the property uncertainty.
- It is measured, not assumed: every term is a set of simulator runs.

- The same band is used everywhere: to choose, to compare PCMs, to judge the dense-grid check and the seed spread.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Statistical t-test / p-value	Only five years per design, and the years are not independent replicates; p-values would be over-confident (the ANOVA notes this) and would miss model uncertainty.
Fixed tolerance, e.g. 5 %	A percentage of what? It would be large in warm regimes and meaningless near zero in regime 3.
No threshold	Any tiny difference would be reported as a 'better' design; the regime-3 result shows why that matters.

4.11 NSGA-II, active learning and the box-vertex step

What it is and what we did

NSGA-II (Deb et al. 2002) searches for the Pareto front of three objectives: maximise g_window, maximise g_annual, minimise d_overheat, with the PCM peak at or below 65 C (Deb's feasibility-first rule; predicted peak + 1 C guard). Population 100, 100 generations, 3 seeds; simulated-binary crossover and polynomial mutation.

Active learning: after each run, the 10 front designs where the three surrogate families disagree most are simulated and added to the training data, and the surrogate is refitted. Stop when the simulated front's hypervolume changes by less than 1 % for two rounds (up to 5 rounds). Then the 24 corners of the design box are simulated and 3 more rounds run (the 'box-vertex step', files *_ext2).

Why we used it

- There are three objectives that conflict (more PCM gives more evening energy but more overheating hours); NSGA-II gives the whole trade-off at once instead of forcing weights in advance.
- It is the standard, well-tested multi-objective genetic algorithm; constraints are handled without penalty weights.
- Written in numpy because pymoo and DEAP were not installed; it is tested on the ZDT1 benchmark and on a constrained problem (tests/test_nsga2.py).
- Active learning corrects the surrogate exactly where it is unsure, with few extra simulator runs.
- The box-vertex step was added because the best designs sit at the edges of the box (40 mm radius, 30 % PCM), where Sobol points are sparse.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Weighted sum of objectives	Needs weights chosen in advance, and cannot find the parts of a non-convex front.
MOEA/D, SPEA2	Equally valid; no reason to expect better results for 3 objectives, and NSGA-II is the most cited choice in PCM-tank papers.
NSGA-III	Designed for 4 or more objectives; for 3 it behaves like NSGA-II.
Bayesian optimisation (expected improvement)	Efficient for one objective; multi-objective versions are complex, and the cheap simulator (1 s per year) removes the main reason to use it.
Particle swarm, simulated annealing	Mainly single-objective; would need weights.
Optimise the simulator directly with NSGA-II	100 x 100 x 3 seeds x 5 years = 150,000 simulated years per pair, 1.8 million for 12 pairs: weeks of computing.

4.12 Dense grid with hypervolume and epsilon indicators (checking the optimiser)

What it is and what we did

For the rank-1 PCM of each regime, a regular grid of 6 radii x 8 PCM fractions x 5 flows x 3 arrangements = 720 designs is simulated for all 5 explore years. The grid's Pareto front is compared with the optimiser's front using the hypervolume ratio and the additive epsilon indicator; the three seeds are compared by their hypervolume spread.

Why we used it

- It is a brute-force test that the surrogate plus NSGA-II did not miss a better region.
- Result: in all 4 regimes the optimiser's front is within the band of the grid front (hypervolume ratio 1.05-1.08), and the seed spread is inside the band in all 12 pairs.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Trust the optimiser without a check	Surrogate errors could hide a better design.
A finer grid	Grows as the 4th power of the resolution; 720 designs is already enough to judge against a band of about 20 kWh.

4.13 Selection rule (S0 - S1 - S2, then lowest PCM mass)

What it is and what we did

On the select years: keep designs that meet the hard limits; S0 = those within the band of the best g_window; S1 = within S0, those within the band of the lowest overheating; S2 = within S1, those within the tau_serve uncertainty of the best. Pick the lowest PCM mass, then the fewest capsules, then the largest margin to 65 C. 'Better than water-only' is claimed only if g_window exceeds its band.

Why we used it

- Written down before the selection was run, so the result cannot be steered.
- Designs the band cannot tell apart are treated as equal, and the cheapest (least PCM) is chosen, which is the sensible engineering choice.
- It gives an honest answer when nothing helps: in regime 3 no design beats the band, so the rule flags 'no meaningful benefit' and recommends water-only.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Take the single best g_window	Picks a winner that may be inside the noise, and ignores overheating and cost.
TOPSIS, LINMAP, knee point	Need weights or a distance measure chosen by the analyst; the result depends on that choice. The band-based rule needs no weights.
Weighted score	Same problem: the weights decide the winner.

4.14 One-shot held-out test, ablations and controls

What it is and what we did

The chosen design of each regime and the best design of each other PCM are simulated once on 2024-2025; a lock file (run id heldout_c75ac0e6fb78) blocks any second run. Ablations change one variable at a time; controls check that the water-only tank does not change with the design, that runs are deterministic, and that removing the latent heat removes the gain.

Why we used it

- A test that can be repeated stops being a test. The lock makes the result trustworthy.
- Ablations and controls prove that the gain comes from the PCM physics, not from a coding effect: with latent heat set to zero, the gain falls by 139-155 kWh in regimes 0-2.
- Result: 76.0 / 61.8 / 56.4 kWh/yr on the test years for regimes 0-2, all above their bands; regime 3 -6.1 kWh/yr.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
Cross-validation only	Every year would influence the choice at some point; there would be no untouched years left.
Testing many designs and reporting the best	Cherry-picking: the test would no longer be fair. The other PCMs' best designs are included to show the comparison, not to choose.

4.15 CFD check: OpenFOAM, enthalpy-porosity, buoyantPimpleFoam

What it is and what we did

One capsule of each chosen design is simulated in OpenFOAM v2406 as a thin 5-degree wedge of a sphere (axisymmetric), with the enthalpy-porosity melting model, Boussinesq buoyancy for natural convection in the melt, and the water-side coefficient and aluminium wall in the boundary condition. Also 12 cases of the 50 L tank (charge, draw, idle) per regime.

Why we used it

- It checks the one simplification that matters most in the simulator: the lumped capsule resistance R_pcm = r/(5k) with the x2 convection factor.
- OpenFOAM is free and scriptable, so all cases are generated from the Objective 2 results automatically; the same scripts serve every state.
- First result (regime 0): CFD 90 % melted in 71 min, simulator 145 min; the simulator is conservative, so the gains are not overestimated.

Alternatives and why we did not use them

Alternative	Why not (for this problem)
ANSYS Fluent	Commercial licence; the student licence limits parallel sessions (Tamil Nadu's experience). Same physics model, so no gain in accuracy.
COMSOL	Commercial; same physics.
Full 3-D sphere	About 50-100 times more cells for little change; the axisymmetric wedge is the standard choice and published work shows it is, if anything, slightly slower than 3-D.
Use CFD inside the optimiser	Minutes to hours per capsule; impossible for thousands of designs.

Status. The Rajasthan OpenFOAM benchmark against published experiments (Tan 2009, Dong 2022) has not yet reproduced the experiments (CFD melts about 2 times faster), so c = 5 and F = 2 are kept and no CFD correction is applied in any state. The Uttarakhand CFD is model-to-model.

5. Running Objective 2 faster: parallel tasks and CPU use

Objective 2 needs tens of thousands of simulated weather-years. This section explains where the computing time goes, what is already done to use all CPU cores, what else can make it faster, and the exact commands. All timings below were measured on this laptop on 9 November 2026 with the Uttarakhand code.

5.1 The machine and where the time goes (measured)

Item	Value	What it means
CPU	Intel Core i7-13700H: 14 cores (6 performance + 8 efficiency), 20 threads	About 14 simulator processes can run at full speed at once; the efficiency cores are slower
Memory	15.7 GB RAM	The real limit on how many processes can run together
One simulated year	1.09 s (multi-node tank, 10 layers, 150 s step)	One design over the 5 explore years = 5.4 s
Inside one year	The time-step loop is already compiled with numba (machine code); about 25 % (0.27 s) is spent reading the weather table row by row only to get the hour of the day	The physics is already fast; the remaining waste is Python/pandas overhead
Memory per simulator process	About 180 MB	12 processes need only about 2-3 GB for the simulator itself; surrogate fitting needs more
Whole chain (baseline to selection)	About 6 h with 6 parallel jobs; active learning alone 193 min	This is the time to beat

Most of the work is **embarrassingly parallel**: the 12 (regime, PCM) pairs never share data, the designs inside a DOE or grid are independent, and even the 5 weather years of one design are independent. Only the order of the steps is fixed, because each step needs the previous step's files.

5.2 Ways to make it faster

Method	What it does	Speed-up	Status
1. Run the 12 pairs as separate processes	One Python process per (regime, PCM) pair; pipeline.py --jobs N	Up to N times for the pair-based steps (S1, S2, S4/S5)	USED (6 jobs). Can go to 10-12 if memory allows
2. Run the 5 weather years of a design in parallel	A process pool inside the design evaluation: one year per worker	Measured 3.8x for one design (5.43 s to 1.42 s, identical result)	NOT in the code. Most useful in steps with only 4 units (band, dense grid, selection, ablations), where most cores would otherwise sit idle
3. Parallel over the designs of one step	A process pool over the DOE or grid designs of one pair (the surrogate comparison already uses 9 workers)	Close to the number of free cores	USED in S3 only
4. Remove the row-by-row weather reading	Read the local_hour column once as an array instead of row by row	About 25 % faster per simulated year (0.27 s saved of 1.09 s), identical values (checked)	NOT in the code; a one-line change, to be checked by the regression tests
5. One thread per process	Set OMP_NUM_THREADS=1 (and MKL / OPENBLAS) so 12 processes do not each start 20 numpy threads	Avoids the slow-down from oversubscription	XGBoost already uses n_jobs=1; set the variables for safety
6. Compile the time-step loop (numba)	Turns the Python loop into machine code	Already done (typically 10-50x versus plain Python)	USED
7. Cache weather and baselines	Each weather year is read once per process; the water-only tank is pre-computed on a flow grid	Avoids repeated file reading and re-running the plain tank	USED (lru_cache, plain_baseline_*.csv)

Method	What it does	Speed-up	Status
8. Laptop power settings	Plug in the charger, choose the Best performance power mode, keep the laptop cool	A laptop CPU on battery or when hot can run at about half speed	Practical, no code change
9. OpenFOAM: several cases at once	Each capsule case is small (about 1,200 cells), so run 4-5 cases side by side, each on one core, on the Linux disk (~/o2_runs)	About 4-5x for the 26 cases	READY (commands in 5.8)

Expected effect if methods 1, 2 and 4 are combined (an estimate, not yet measured end to end): the pair-based steps run 12 at once instead of 6, the 4-unit steps use 4 x 5 workers instead of 4, and every simulated year is about 25 % shorter. That would bring the 6-hour chain down to roughly 2-3 hours.

5.3 What does not help, and why

Idea	Why not
GPU (graphics card)	The simulator steps through time one step after another (each step needs the previous one), and each step is a tiny 10 x 10 system. GPUs are fast only for very large parallel calculations. The surrogates train in seconds on about 200 points, so a GPU gives nothing there either.
More than 14 simulator processes	The 20 threads come from 14 physical cores; extra processes share cores and each runs slower. Memory (15.7 GB) runs out first.
Splitting one OpenFOAM capsule over several cores (MPI)	With about 1,200 cells the communication between cores costs more than it saves; running several cases side by side is faster.
A larger time step	300 s failed the convergence check in Rajasthan; 150 s is the largest verified step.
Fewer weather years or fewer designs	Faster only because it does less work: it changes the method and weakens the result.

5.4 How method 2 would be written (proposed code, not yet in the repository)

A pool of worker processes evaluates the years of one design at the same time; the per-year results are then combined exactly as the serial loop does. It must give results identical to the serial version (checked by a test) before it is used.

```
from concurrent.futures import ProcessPoolExecutor
from src.optimize.evaluate import evaluate_design

def _one_year(args):
    design, year, purpose = args
    return evaluate_design(*design, purpose=purpose, years=[year])

def evaluate_design_parallel(design, years, purpose="explore", workers=5):
    with ProcessPoolExecutor(workers) as ex:
        parts = list(ex.map(_one_year, [(design, y, purpose) for y in years]))
    # then combine the per-year results exactly as evaluate_design does
    return parts
```

> **None of these code changes is needed for the results already reported.** The Uttarakhand results are final and the held-out test has been used; methods 2 and 4 matter only if the chain is ever re-run (for example for another state, or after a change in Objective 1). They must be added before such a re-run and verified with `python -m pytest tests`.

The rest of this section is the practical part: which steps can run in parallel, and the commands.

5.5 What can run in parallel

Step	Script	Independent units	Recommended at once
Water-only baseline (G8)	g8_build_plain_baseline.py <regime>	4 regimes	4
S1 Taguchi screening	g8_s1_screening.py <regime> <rank>	12 pairs	6

Step	Script	Independent units	Recommended at once
S2 Sobol DOE	g8_s2_doe.py <regime> <rank>	12 pairs	6
S3 surrogate comparison	g8_s3_compare.py	one script, 9 workers inside	run once
WP6 band	g9_band.py <regime>	4 regimes	4
S4/S5 active learning	g9_s4_s5.py <regime> <rank>	12 pairs	6
S4/S5 box-vertex step	same, with G9_WARM=1 G9_VERTICES=1	12 pairs	6
S6 dense grid	g9_s6_grid.py <regime> 1	4 regimes	4
S7 selection	g10_select.py <regime>	4 regimes	4
G11 ablations	g11_ablation.py <regime>	4 regimes	4
OpenFOAM capsule cases	run_one.sh <case>	10 cases	4-5
OpenFOAM tank cases	tank_run.sh <case>	12 cases	4

Steps must still run in order (screening and DOE need the water-only baseline, active learning needs the DOE and the surrogate comparison, selection needs the optimiser's fronts); only the units inside a step run at the same time.

> **Before re-running anything that decides the designs, read this.** The held-out test has been used (`results/held_out_test.lock`). Re-running baseline, screening, DOE, surrogate, band, optimise or select overwrites the final results and makes the held-out test meaningless. `pipeline.py` refuses these stages while the lock exists unless `--allow-invalidate` is given; calling a script directly bypasses that protection. Back up `results/` first, and never run `g10_test.py` again. Reporting stages (status, report, handoff, plots) and the OpenFOAM cases are always safe.

5.6 Recommended: the pipeline's own parallel runner

`pipeline.py` already runs the units of each stage in parallel (`--jobs`) and the stages in order. From `objective2-uttarakhand`:

```
# safe at any time
python pipeline.py --state uttarakhand --stage status
python pipeline.py --state uttarakhand --stage optimize --dry-run      # print the commands only

# re-running a deciding stage (only after backing up results/)
Copy-Item -Recurse results results_backup
python pipeline.py --state uttarakhand --stage doe --allow-invalidate --jobs 6
python pipeline.py --state uttarakhand --stage optimize --allow-invalidate --jobs 6
```

5.7 The same steps by hand in PowerShell (12 pairs)

Run from `objective2-uttarakhand`. `Start-Process` returns at once; wait until the count of Python processes drops to 0 before starting the next step. These commands start all 12 at once; on this machine that is possible, but 6 at a time is safer for memory, so the pairs are split into two halves.

```
# one numpy thread per process (method 5)
$env:OMP_NUM_THREADS = "1"; $env:MKL_NUM_THREADS = "1"; $env:OPENBLAS_NUM_THREADS = "1"

$pairs = foreach ($r in 0,1,2,3) { foreach ($p in 1,2,3) { "$r $p" } }
function Run-Pairs($script, $tag, $list) {
  foreach ($rp in $list) { $n = $rp -replace ' ', '_p'
    Start-Process python -ArgumentList "scripts/$script $rp" -NoNewWindow `
      -RedirectStandardOutput "results/log_${tag}_r$n.txt" -RedirectStandardError
"results/err_${tag}_r$n.txt" } }

# Step 0: water-only baseline grids, 4 regimes (explore and select years)
foreach ($r in 0,1,2,3) { Start-Process python -ArgumentList "scripts/g8_build_plain_baseline.py $r"
-NoNewWindow -RedirectStandardOutput "results/log_base_r$r.txt" }

# Step 1: Taguchi screening (S1), first 6 pairs, then the other 6
Run-Pairs g8_s1_screening.py s1 $pairs[0..5]     # wait, then:
Run-Pairs g8_s1_screening.py s1 $pairs[6..11]
```

```
# Step 2: Sobol DOE (S2), same pattern, then repair any memory errors
Run-Pairs g8_s2_doe.py s2 $pairs[0..5]          # wait, then $pairs[6..11]
python scripts/g8_s2_repair.py

# Step 3: surrogate comparison (uses 9 workers itself)
python scripts/g8_s3_compare.py

# Step 4: band, 4 regimes
foreach ($r in 0,1,2,3) { Start-Process python -ArgumentList "scripts/g9_band.py $r" -NoNewWindow -
RedirectStandardOutput "results/log_band_r$r.txt" }

# Step 5: active learning (S4/S5), then the box-vertex step (rounds = start + 3, set by the script)
Run-Pairs g9_s4_s5.py g9 $pairs[0..5]           # wait, then $pairs[6..11]
$env:G9_WARM = "1"; $env:G9_VERTICES = "1"
Run-Pairs g9_s4_s5.py g9ext2 $pairs[0..5]       # wait, then $pairs[6..11]
Remove-Item Env:G9_WARM, Env:G9_VERTICES

# Step 6: dense grid, rank-1 PCM, 4 regimes; then the checks
foreach ($r in 0,1,2,3) { Start-Process python -ArgumentList "scripts/g9_s6_grid.py $r 1" -
NoNewWindow -RedirectStandardOutput "results/log_s6_r$r.txt" }
python scripts/g9_assess.py

# Step 7: selection, 4 regimes
foreach ($r in 0,1,2,3) { Start-Process python -ArgumentList "scripts/g10_select.py $r" -NoNewWindow
-RedirectStandardOutput "results/log_g10_r$r.txt" }

# Progress
(Get-Process python -ErrorAction SilentlyContinue | Measure-Object).Count
Get-Content results/log_g9_r0_p1.txt -Tail 5
```

5.8 OpenFOAM cases in parallel (safe now)

Each run script copies its case to its own folder in `~/o2_runs`, so several cases can run at once. Each OpenFOAM case uses one core; 4-5 at a time leaves the machine usable. In WSL bash:

```
U=/mnt/d/Objective---2/objective2-uttarakhand/openfoam_cfd

# the 9 capsule cases not yet run, 5 at a time
ls -d $U/cases/*/ | grep -v 'r0_chosen_puretemp_58/$' | xargs -P 5 -I{} bash $U/scripts/run_one.sh
{}

# then post-process them (Windows Python, from the scripts folder)
cd $U/scripts
for c in ../cases/*/; do python.exe postprocess.py "$c" --window-h 6; done

# the 12 tank cases, 4 at a time (each about 75 min per simulated hour)
ls -d $U/cases_tank/*/ | xargs -P 4 -I{} bash $U/scripts/tank_run.sh {}
for c in ../cases_tank/*/; do python.exe tank_postprocess.py "$c" --T-target 50; done

# the 4 animation cases, 4 at a time
ls -d $U/cases_viz/*/ | xargs -P 4 -I{} bash $U/scripts/viz_run.sh {}

python.exe summarise_results.py
```

6. The final output of Objective 2

Regime	Chosen design	Gain on select years	Gain on test years 2024-25	Verdict
0	PureTemp 58, R 39.8 mm, 26.3 % PCM, 47 capsules, 0.019 kg/s, staggered	80.0 kWh/yr	76.0 kWh/yr	PCM helps
1	PureTemp 58, R 39.3 mm, 29.7 %, 55 capsules, 0.031 kg/s, single-layer	72.1	61.8	PCM helps
2	savE OM55, R 39.7 mm, 30.0 %, 54 capsules, 0.020 kg/s, single-layer	63.0	56.4	PCM helps
3	(n-Hexacosane, 2 %, 4 capsules: least-PCM design)	-5.8	-6.1	Use a water-only tank

All designs stayed safe: PCM at most 64.2 C (limit 65), water at most 73.1 C (limit 75).

The deliverables are:

1. **The chosen design per regime** with the evidence: `results/g10_selection_r0..r3.json` and the held-out results `results/g10_heldout_results.csv`.
2. **The Objective 3 hand-off contract** `results/obj3_environment_contract_uttarakhand_v2.json`: simulator settings, state and actions, limits, weather split, the chosen designs, bands and a reward suggestion, for the controller that Objective 3 will build.
3. **The documents**: the main report `docs/Objective2_Uttarakhand_Report_v3.docx`, the docs 01-06, this document, the figures, and `results/results_registry.csv`, which links each headline number to its source file.
4. **The CFD check** in `openfoam_cfd/` (1 of 10 capsule cases run so far).

7. Summary table

Step	Used	Main reason	Main alternative not used
Simulator	Grey-box physics model	1 s per year, traceable equations	CFD (too slow), TRNSYS
PCM	Enthalpy method	Conserves latent heat exactly, always converges	Apparent heat capacity
Tank	10-layer stratified	Hot water at the top, as in real tanks	Single mixed node
Time step	Linear-implicit, 150 s	Stable and verified (0.03 %)	Explicit, Runge-Kutta
Comparison	Paired same-flow water-only tank	Isolates the effect of the PCM	Absolute energy
Years	Explore / select / one-shot test	No selection bias	Use all years
Screening	Taguchi L9 + ANOVA	9 runs instead of 81	Full factorial
DOE	Scrambled Sobol, 128	Even coverage with few runs	Random, Latin hypercube
Surrogate	XGBoost / RSM / NN, chosen by rank validation	Data-chosen, never decides results	Kriging, RBF, SVR
Real-gain test	Measured WP6 band	Includes model uncertainty	p-values, fixed %
Optimiser	NSGA-II + active learning + box vertices	3 conflicting objectives, no weights	Weighted sum, BO
Check	Dense grid, hypervolume, epsilon	Proves no better region was missed	No check
Choice	Band-based lexicographic rule	Pre-declared, no weights, honest 'no gain'	TOPSIS, best-only
Final check	One-shot held-out test + lock	An honest result	Cross-validation only
Physics check	OpenFOAM enthalpy-porosity	Free, scriptable, same physics as Fluent	Fluent, COMSOL
Speed	Parallel processes per pair, numba-compiled loop, caching (years in parallel proposed)	Uses the CPU cores; chain about 6 h	GPU (no benefit for this model)

Source files for every statement: `docs/01-06`, `docs/PROGRESS.md`, the scripts named above and `results/`.